Revision History

Date	Version	Description
October 11, 2019	0.50	Initial version created.
February 14, 2020	0.51	Updated sections Bulk, Write Data Start, Write Data Continuation, Last Write Status, Write Data; updated Table 10, 11; added section Write Data Stream Mode.
May 11, 2020	0.52	Updated Table 3, Table 12, Installing the profile; added Locking and Unlocking Sessions, Version String, Configuring the Flash Size, Cleaning Upgrades.
December 2, 2020	0.53	Updated Cleaning Upgrades, corrected typos.
April 16, 2021	0.54	Updated format, no content change.

CONFIDENTIAL

©2019-2021, Atmosic Technologies Inc. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc.
All other trademarks are the properties of their respective holders. This document is subject to change without notice.

Table of Contents

References 5

Glossary 5

Overview 6

Features 6

Design 7

Architecture 7

Client Profile 7

Server Profile 8

Upgrade Processor 9

Flash Subsystem 9

Flash 9

Flash Boot 10

Upgrade State 10

Upgrade Process Flow 11

Use of a CRC/Hash 12

Upgrade Flow Step Details 13

First Boot Watchdog 14

Messages 15

Types 15

Command 15

Status 15

Completion 15

Write Data 15

Ports 15

Mailbox 15

Bulk 16

Definitions 16

Message Format 16

Status Codes 16

Command Completion 18

Cancel 19

Bluetooth LE Considerations 19

Message size limits 19

Pending Message Indication 20

Locking and Unlocking Sessions 20

Lock Session 21

CONFIDENTIAL

Unlock Session 21
Query Information 21
Query Version Information 23
Write Data Start 24
Write Data Continuation 25
Copy Data 26
Last Write Status 27
Read Data 27
Get Programmed State 28
Erase Sectors 29
Set Upgrade Status 30
Reboot 31
Message Flows 31
Control 32
Write Data 33
Write Data Stream Mode 34

Developers Guide 36
Installing the profile 36
Changing the Service UUID 37
Configuring the Flash Size 37
Cleaning Upgrades 37

CONFIDENTIAL

List of Figures

Figure 1 - OTA System Architecture Diagram
Figure 2 - 512 KByte Flash Partitioning
Figure 3 - Upgrade Process Flow Diagram
Figure 4 - Upgrade Process Flow Diagram con't
Figure 5 - Message Flow Diagram
Figure 6 - Mailbox Port Write Data Flow
Figure 7 - Dropped Message in Write Buffer
Figure 8 - Bulk Port Write Data Flow

List of Tables

Table 1 - Server Profile
Table 2 - Message Format
Table 3 - Status Codes
Table 4 - Command Status
Table 5 - Command Completion
Table 6 - Cancel Command
Table 7 - Message Size and Parameter Length Limits
Table 8 - Pending Message Indication
Table 9 - Lock Session
Table 10 - Unlock Session
Table 11 - Query Information
Table 12 - Query Status Parameters
Table 13 - Query Version Information
Table 14 - Status Message
Table 15 - Write Data Start
Table 16 - Write Data Status Parameters
Table 17 - Write Data Completion Parameters
Table 18 - Write Data Continuation
Table 19 - Copy Data
Table 20 - Last write status
Table 21 - Status message
Table 22 - Read Data
Table 23 - Read Data Status Parameters
Table 24 - Get Programmed State
Table 25 - Range Descriptor
Table 26 - Programmed State Status Parameters
Table 27 - Erase Sectors
Table 28 - Erase Sectors Descriptor
Table 29 - Set Upgrade Status

CONFIDENTIAL

Table 30 - Reboot

References

Atmosic SDK E3c User Reference Manual, Flash Update section
Bluetooth Specification

Glossary

Acronym	Description
ATT	Attribute Protocol
BULK	DTS bulk data port
DTS	Atmosic Data Transfer Service Characteristics
GATT	Generic Attribute
MBOX	DTS mailbox data port
MTU	Maximum Transmission Unit
OTA	Over the Air
RFU	Reserved for Future Use

CONFIDENTIAL

Overview

Over the Air (OTA) updates allow end users to update their systems in the field without the need for wired connections. A mobile application can deliver firmware updates wirelessly as part of the device onboarding process or a regularly scheduled update process. This document describes the Atmosic proprietary OTA update Bluetooth LE profile (server/peripheral role) and messaging interface to allow customers to manage upgrades through mobile or desktop client applications.

Features

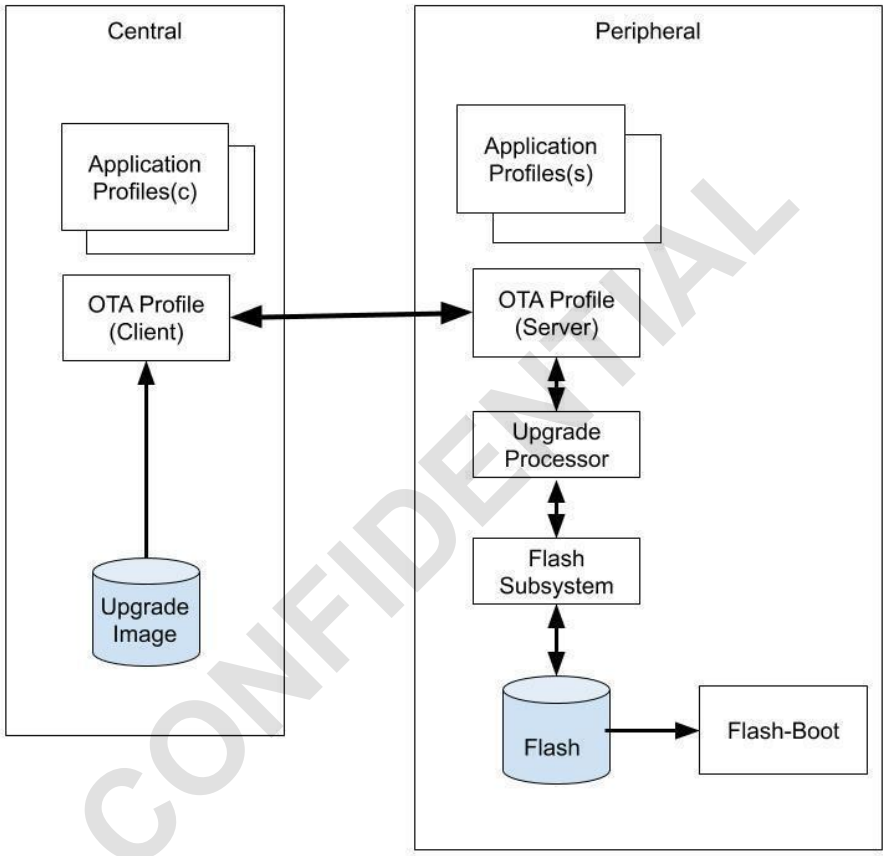
- OTA server profile with customizable 128-bit service UUID.
- Server profile can coexist with other profiles. Profile can be installed dynamically or statically at boot time.
- Low memory footprint with application configurable write buffer size.
- Support for Atmosic Flash partition upgrade management (see SDK E3c User Reference Manual)
- Support for optional first boot watchdog mechanism
- Flexible OTA command set to develop both simple and complex (background update) client applications.
- Allows resumed updates from connection or power cycling interruptions.
- Standard Bluetooth LE link security and authentication
- Methods to inspect, copy (transfer) and write NVDS

CONFIDENTIAL

Design

Architecture

The OTA system is described in the following diagram:



CONFIDENTIAL

Figure 1 - OTA System Architecture Diagram

The central is the mobile or desktop host running the end-user application. Atmosic provides client libraries for common operating systems to rapidly integrate OTA capabilities into your application. The following section provides details on each of the major components.

Client Profile

The client profile is implemented in the central's Bluetooth LE stack (mobile or desktop stack). The profile is GATT- based and will use GATT write and read procedures to

communicate with server-side characteristics. The client interface to the OTA system consists of DTS ports that implement communications ports tailored for command/response and data.

Server Profile

The server profile implements the OTA upgrade message command set and data transfer protocols. The server leverages the data transfer service (DTS) ports and data flow behavior and requires the following port types:

1 bulk port (optional fast write data transfers) - 512 bytes
1 mailbox port (control/status/data) - 512 bytes

The default service UUID is: a0e78d39-75b5-4182-8fdc-c4b7365c9062
Customers can reconfigure the profile to install their own UUID.

Table 1 - Server Profile

Attribute	Attribute Type	Permissions	Values	Notes
Service	Primary Service Declaration	RO	Service UUID defined by the user	No include declaration needed. UUID: default ATM OTA
Bulk Port Char	Characteristic Declaration	RO	Properties, Value Handle, UUID	UUID: 040C6507-F08A-4FC2-ABEA-6382EE250230
Bulk Port Char Value	Characteristic Value Declaration	Write No-RSP / Notify	512 -byte value size	Bulk Data Flow (Stream). Max transfer ATT_MTU - 3
Bulk Port CCC	Client Characteristic Configuration Descriptor	R/W	CCC value	Set value to 0x0001 to enable ATT notifications
MBOX Port Char	Characteristic Declaration	RO	Properties, Value Handle, UUID	UUID: CB91B0D6-3080-4DFB-876B-407DB045E52B
MBOX Port Char Value	Characteristic Value Declaration	Write / Indicate	512 -byte value size	Mailbox Data Flow. Max read/write of 512 bytes. Max indicate is ATT_MTU - 3
MBOX Port CCC	Client Characteristic Configuration Descriptor	R/W	CCC bit	Set value to 0x0002 to enable ATT indications

CONFIDENTIAL

Upgrade Processor

The upgrade processor is a reusable library that implements the flash upgrade command set. There is only one instance of the upgrade processor in the system and it will be initialized and managed by the server profile. The Bluetooth LE server profile provides an over-the-air messaging channel that bridges the client to the upgrade processor. For most commands the Bluetooth LE client is interacting mainly with the upgrade processor. The upgrade engine interfaces with the flash subsystem to access and manipulate the flash partitions.

Flash Subsystem

This consists of the SPI controller, external flash drivers (see SDK), and partition management APIs (also part of the flash drivers).

Flash

This is the SPI-NOR flash device (multiple vendors) attached to the SPI bus controller. Please refer to the flash drivers and documentation for the supported types. Flash is partitioned via software controlled programming of the AHB bus control hardware to map either the upper or lower portions of the flash device into the system's designated flash address memory space (0x10000000). The following diagram illustrates the partitioning using a 512 KB flash part. Regardless of flash size, the partitioning is always binary. The term ***bank*** and ***partition*** are interchangeable in this document.

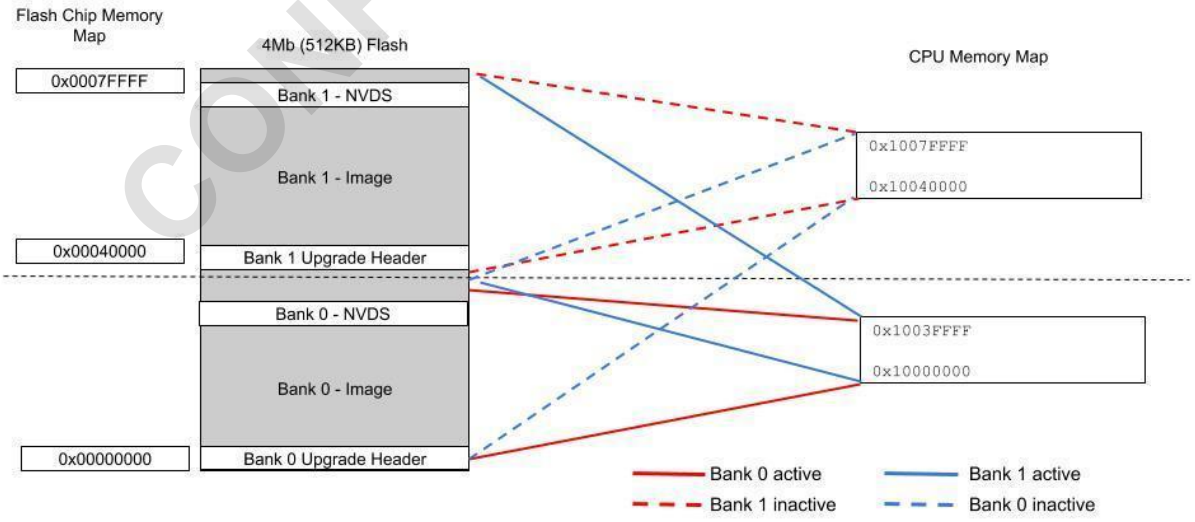


Figure 2 - 512 KByte Flash Partitioning

CONFIDENTIAL

Flash Boot

This is the flash boot process in ROM. The ROM determines which partition is valid and maps that partition into the address space of the CPU. For more details, please refer to the SDK user manual listed in the references.

Upgrade State

The server profile does not maintain state of the flash upgrade process. Upgrade session state is not maintained between re-connections or power/reset cycles. Any upgrade state can be determined through an inspection of the inactive partition stored data. The OTA profiles provide methods to aid in assessing the programmed state of the inactive partition.

CONFIDENTIAL

Upgrade Process Flow

The high level image upgrade flow is described in the following diagrams. ***Note: Blocks that are enumerated (1.,2.) have a corresponding detailed description in the Upgrade Flow Step Details section.***

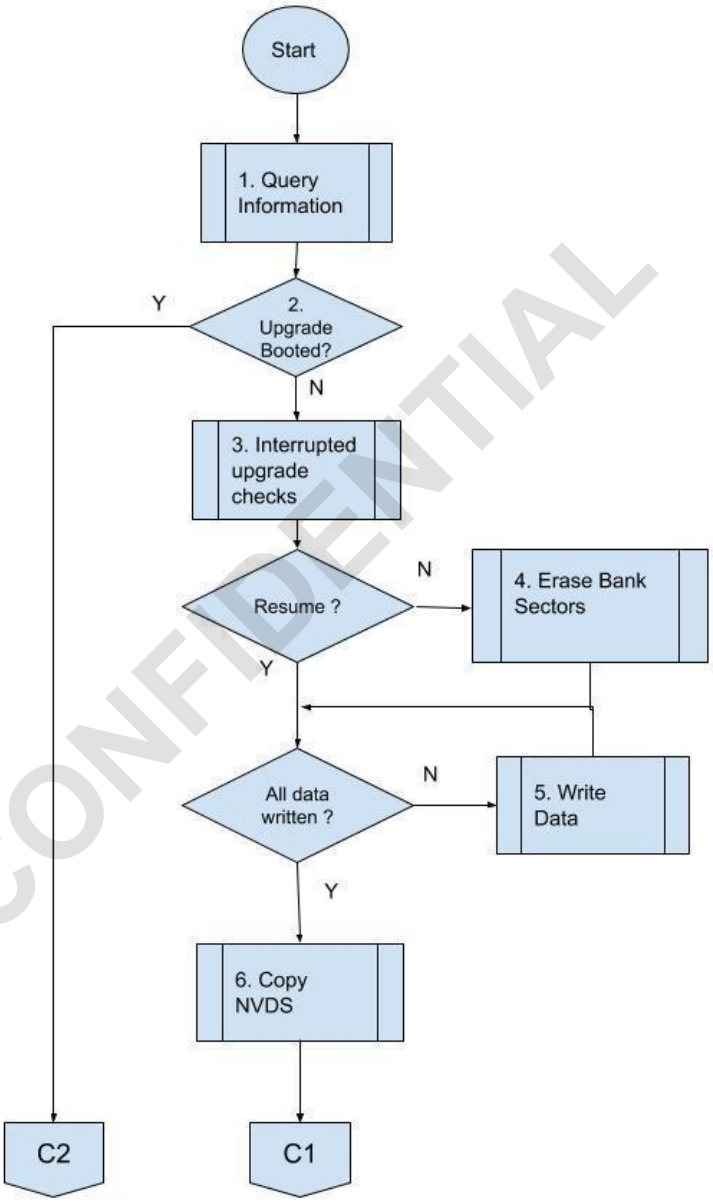
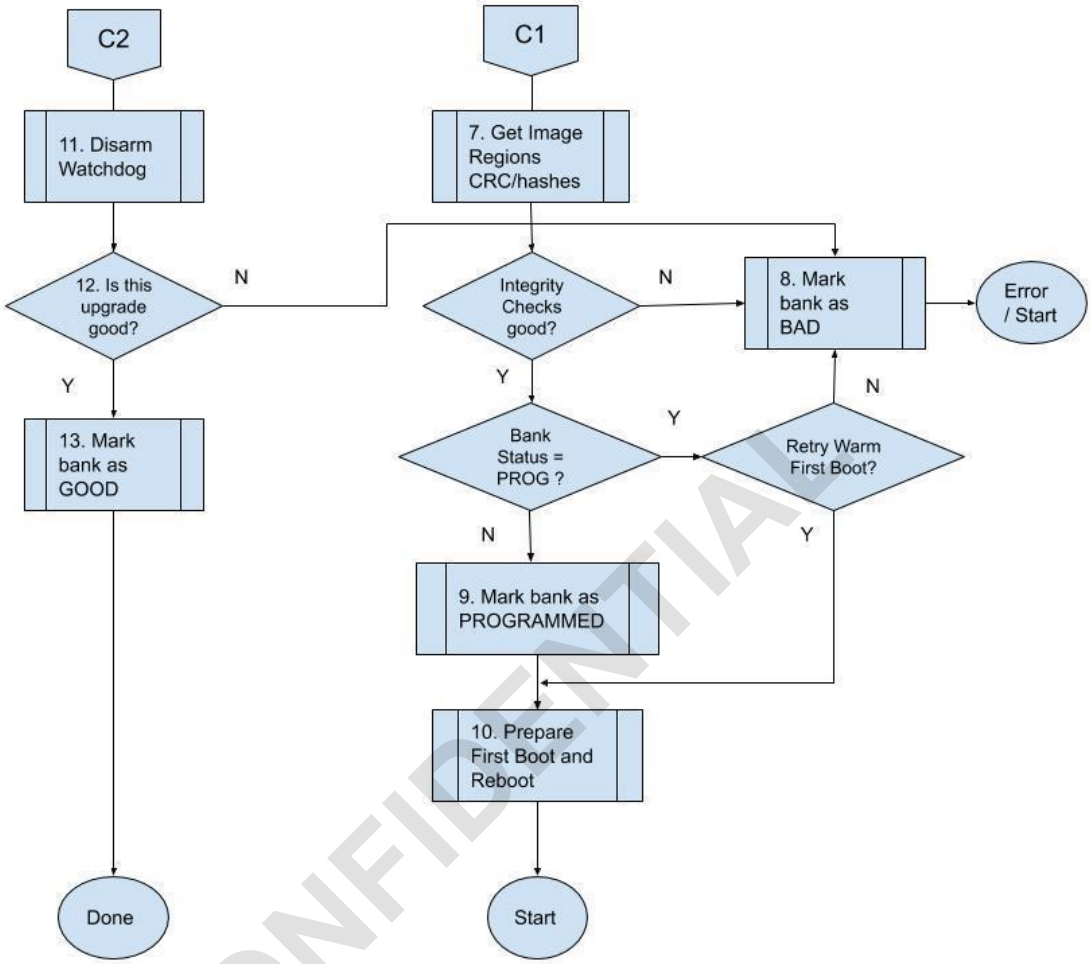


Figure 3 - Upgrade Process Flow Diagram

CONFIDENTIAL



CONFIDENTIAL

Figure 4 - Upgrade Process Flow Diagram con't

Use of a CRC/Hash

The upgrade process makes heavy use of CRC codes to verify the integrity of the data written to flash. This avoids costly data reads to verify flash programming. Data in transit however is not CRC/hashed as the transport can provide data integrity checks in transit. Ultimately the determinant of programming success and failure rests on what is actually programmed. The profile provides commands to CRC/hash multiple regions disjointly with one command. An optional erase-verify operation is also provided to ensure that sectors are properly erased. The CRC selected is CRC-32 for implementation simplicity, relative speed and error detection.

Upgrade Flow Step Details

The enumerated steps in the previous diagrams are listed below.

1. Client connects to the server profile and queries information regarding the platform and upgrade state of the system.

2. The upgrade state of the active and inactive partition are checked to determine if the client should perform a new upgrade or resume an interrupted upgrade. An interrupted upgrade can be determined if the inactive partition has an upgrade status of NOTPROGRAMMED or PROGRAMMED:
    a. If PROGRAMMED the inactive partition did not first boot. The client can re-check CRC and retry a warm reboot again; otherwise fail and mark the image as BAD.
    b. If NOTPROGRAMMED is detected the partition is either new (new device) or a prior erase or programming operation was interrupted. Note that on a brand-new device (first time programmed at the factory), the inactive partition will always show the status of NOTPROGRAMMED. The client can go to step 3 to check for an interrupted session.
    c. In all other cases the inactive partition was upgraded before (old) or upgraded and marked BAD. In this case a new upgrade should be performed.

3. In an interrupted upgrade, the client determines whether to erase or resume writing data. If the client wishes to optimize this process, they have the option to employ their own algorithms to determine where the write progress left off. This is done with a combination of CRC commands and erase-verify commands to "walk" the inactive partition. The client can get a reasonable assessment of where it left off. Otherwise the client can just start fresh and erase the partition. If an erase operation is interrupted (disconnect or system reset), the client is required to determine where it left off when it reconnects and should make no assumptions about whether the last operation succeeded or not. ***The client should also take steps not to excessively erase programmed sectors due to an interrupted upgrade. Repeated erasure/programming reduces the endurance of flash device.***

4. Any image update requires flash sectors to be erased prior to writing. Since erasure takes time, the erase requirement is separate from writes. The client manages where the inactive partition is erased and how many sectors need to be erased. The erase operation can perform a final erase-verify across the sectors if desired.

5. Data blocks of the image are written to staging buffers and then buffers are written to flash. If a write operation is interrupted (disconnect or system reset), the client is required to determine where it left off when it reconnects and should make no assumptions about whether the last operation succeeded or not.

6. When the last data block is written, the client should then transfer NVDS data from the previous partition to the new partition. On an interrupted upgrade, determining whether NVDS was copied via CRC checks is problematic due to the dynamic nature of NVDS (its data can change since the upgrade was interrupted). On an interrupted upgrade the client should erase-verify the NVDS sectors in the inactive partition and erase the sector again if not erased. Then the client either requests a copy operation or the client can

CONFIDENTIAL

read the NVDS in the active partition, modify as needed and write the final contents to the inactive partition. The server profile can only report the NVDS location of the active partition (the current running image). The client manages where to write the NVDS in the inactive partition hosting the new upgrade image (either copy or explicit write).

7. The client should request CRC/hash values for memory regions that fully validate the upgrade (not including NVDS). It is recommended that the regions should cover 4-32K blocks to help debug an erase or write problem (***note: it would also help if an image file contained meta data with these regions pre-computed with start/length/CRC values***). Regions such as NVDS can be skipped since this is not part of the upgrade image. The upgrade header region can also be included since its erase state should match what was built into the image (see SDK user reference manual). The sequence and status flags are not programmed until step 9. A client must determine the integrity of flash programming in order to continue with the upgrade (steps 8 and 9)

8. If the integrity check fails the client should mark the partition BAD and retry if required.

9. At this step the client has determined that all flash memory regions have been properly programmed. It is now ready to finalize steps to boot the new upgraded partition

10. At this step the partition is marked as PROGRAMMED however the image is not considered GOOD until the client has had a chance to request a first boot of the new image. The client needs to invoke a special warm reboot process where the ROM can bypass checks and automatically load the newly programmed partition. The client requests a “first boot” reboot. The system will disconnect and reboot.

11. A new upgrade has booted through the mechanism in step 10. The client now determines whether the upgraded image is running properly. Note that the upgraded partition here has been swapped and is now the active partition.

12. This step should ONLY be done when the upgraded partition has been swapped to the active partition through a warm boot (step 10, 11). If the active partition meets the client’s criteria for a good boot, then the client can now mark the partition as GOOD. The client can perform this last step after reconnecting to the OTA server on the new image. Reconnecting to the OTA server is a good indication that the new image is booting properly however the client should make other determinations. A command is issued to mark the current active partition (first booted) as GOOD. In the event the client cannot connect back to the OTA server there is an optional first boot watchdog mechanism.

First Boot Watchdog

The server profile can be configured with a first boot watchdog. This watchdog can be enabled if the active partition is detected as a first boot. The client has until the watchdog timeout to connect and perform a query information otherwise the watchdog will reboot the platform and the ROM will start the last known GOOD partition. The watchdog timeout is typically 30 seconds.

CONFIDENTIAL

Messages

Types

The upgrade protocol uses the following 4 types of messages:

Command

A control message with variable length parameters. Length of parameters depends on the message. A command always results in a status message with an optional completion message for long running commands.

Status

Immediate status reporting of the last command. The status message may also contain a variable number of parameters which are specific to the command. For example read data or area CRC results.

Completion

Completion messages indicate the completion of long running commands (for example: erase) that report final status separately from a status message. Whereas the command status indicates a command is accepted (valid parameters) with PENDING the completion message reports the final result of the operation.

Write Data

Write messages follow the same format as commands however their delivery and status follow a slightly different flow. Write data messages are fragmented messages to post data to a server-side staging buffer (not the flash chip). One or more write data messages will have a single corresponding status and a final completion response. A status is returned on bad parameters, missing fragment (after some timeout) or on a successful transfer right before the programming operation is started. A PENDING status is an indication that the client can now move more data to another staging buffer while programming is in progress. The client can also expect a completion event when programming is complete (either success or failure).

Ports

The following section describes the two port types used for all communications with the server profile. These ports are defined by the Atmosic Data Transfer Service and also used in the Throughput testing profile.

Mailbox

The mailbox is reserved for all command, status and completion messages. Transfers to/from the mailbox use the write-with-response and indicate attribute access procedures. This form of transfer, although reliable and avoids potential resource contention issues, is intended for infrequent message exchanges (i.e. control/status). The control protocol is designed such that most messages can fit into a

CONFIDENTIAL

single ATT transfer procedure (single ATT PDU with confirmation down to an MTU size of 23 bytes) however some messages may exceed the indication size limit (ATT_MTU - 3) and require a client initiated read long ATT procedure. Write data transfers to the flash write buffers can also be performed through the mailbox. This can simplify implementations that do not choose to implement the retransmit logic required over the BULK port.

Bulk

The bulk port provides the fastest and most efficient way to transfer data to waiting write buffers. Data message fragments are sequentially streamed into the write buffers; however, some fragments could be dropped (considered uncommon in most cases). To maximize the performance through the bulk interface, flash programming and staging buffer transfer should be overlapped when possible.

Definitions

Message Format

All fields follow little-endian ordering. For Bluetooth LE, the maximum length of a message is 512 bytes.

Table 2 - Message Format

Size	Field	Description
1	Msg ID	Message ID is 1 octet (1-255), ID 0 is reserved
n	<parameters>	Variable length parameters (See Message size limits)

All messages are encapsulated into the transport's data framing mechanism and thus a length field is not required. The transport layer conveys the length of the full message after decapsulating it and removing any padding. Transport layer is expected to deliver single unfragmented messages.

Status Codes

Table 3 - Status Codes

Name	Value	Description
SUCCESS	0	Success
PENDING	1	Success but the operation is pending and will complete with a completion message
INV_PARAM	2	One or more parameters of the command are invalid

CONFIDENTIAL

UNSUPPORTED	3	Unsupported command or operation
WRITE_FAILED	4	Failure of a write operation. This status is also used by operations that perform data checks on the integrity of the write (see Copy Data and Set Upgrade Status).
ERASE_FAILED	5	Erase operation failed.
BUSY	6	Operation could not be completed because the resource is busy.
MISSING_FRAG	7	One or more fragments are missing in the data transfer request
NOT_ERASED	8	The erase operations's post verify operation detected a non-erased byte or a command could not execute because a location was not erased.
INV_PART_STATUS	9	The server encountered an invalid partition status for the requested operation (see Set Upgrade Status)
WRITE_BUF_OVERFLOW	10	The last write data command has overflowed the write buffer
RESP_OVERFLOW	11	The resulting status or completion response overflows the maximum length allowed by the transport. This can happen if the client requests more read data than the transport can deliver in one response. See (Message size limits)
CANCELLED	12	Operation was cancelled
RD_FAILED	13	Read operation failed
INVALID_REQ	14	Invalid Request

A command status message conveys the success or failure of the last command message. For a command that may require a completion, a PENDING status message indicates the operation has been accepted (parameters validated) and started. Some command processing ends with a status while others may require both a status followed by a separate completion message (i.e. a long running operation completes).

Table 4 - Command Status

Size	Field	Description
1	Msg ID	0x01
1	Cmd Msg ID	The message ID of the command this status is for

CONFIDENTIAL

1	status	Command processing status code
N	<parameters>	Variable length status parameters (See Message size limits)

Command Completion

A command completion message conveys the success or failure of an operation that requires a delayed indication (long running operation). A long running command returns STATUS_PENDING in the status message and provides a final completion status in a completion message shown below:

Table 5 - Command Completion

Size	Field	Description
1	Msg ID	0x02
1	Cmd Msg ID	The message ID of the command.
1	status	Final status of the completed long-running operation
N	<parameters>	Variable length completion parameters (See Message size limits)

CONFIDENTIAL

Cancel

This command cancels the current pending operation.

Table 6 - Cancel Command

Size	Field	Description
1	Msg ID	0x0F

This message returns a status message with no parameters. The PENDING status will indicate that the cancellation is pending. An operation that was cancelled indicates a completion status of CANCELLED. If there is no pending operation at the time the command is received the status returns SUCCESS and no command completion will be issued

Bluetooth LE Considerations

Message size limits

For Bluetooth LE GATT transports, the message size and parameter length limits are listed below:

Table 7 - Message Size and Parameter Length Limits

Message Type	Mailbox Port		Bulk Port	
	Max. Total Length	Max. Parameters Length	Total Length	Parameters Length
Command	512	511	N/A (not exchanged on this port)	
Status	512	509	N/A	
Completion	512	509	N/A	
Read	See status message, the maximum read length is limited by the maximum parameter length.		N/A	
Write Data Start	Same as command		ATT_MTU - 3	ATT_MTU - 4

CONFIDENTIAL

Write Data Continuation	Same as Command	ATT_MTU - 3	ATT_MTU - 4

ATT_MTU - Attribute Maximum Transmission Unit (see Bluetooth Specification)

Pending Message Indication

For Bluetooth LE GATT implementations any message that exceeds the size allowed for an indication (ATT_MTU-3) requires the client to read the message from the port using a read long attribute procedure. To indicate that the client must use this procedure the following message is defined.

Table 8 - Pending Message Indication

Size	Field	Description
1	Msg ID	0xF0
2	length	The length of the pending message that requires a read long procedure

This is typically used when the command status or completion exceeds the length that is allowed in an attribute indication.

Locking and Unlocking Sessions

Bluetooth LE applications can enter operating states that can interrupt OTA updates. For example, a device may disconnect all links in preparation for low power mode. Devices in low power modes may require external stimulus or user intervention to resume normal operations where connections are permitted. Applications can implement out-of-band techniques (switches, buttons) that place devices in OTA friendly states to ensure a timely update and thus improves the user experience. In the absence of such techniques an over-the-air signaling method can be used to lock a connection that is hosting an OTA session in order to prevent the device from entering states that disrupt the upgrade process. The LOCK/UNLOCK session message signals the client's intent to perform an uninterrupted OTA upgrade. This message is propagated as a software notification to the application. The details of how the application processes the locked/unlocked state is beyond the scope of this document. In general the locked state of the session will prevent the system from entering a state that interrupts the update process. The client can unlock the session and allow the system to return to normal operation upon completing upgrade tasks. Additionally a disconnect as a result of the client or the environment automatically unlocks the session.

The client can signal locking and unlocking a session using the following messages :

CONFIDENTIAL

Lock Session

The client can lock the current session to avoid interruptions to the upgrade process. This command returns a status message with no parameters. If the session is already locked the message will fail with INVALID_REQ.

Table 9 - Lock Session

Size	Field	Description
1	Msg ID	0x12

Unlock Session

The client can unlock the current session and return the system to normal operation. This command returns a status message with no parameters. If the session is not locked the message will fail with INVALID_REQ. A session is automatically unlocked if the session's connection terminates while it is locked.

Table 10 - Unlock Session

Size	Field	Description
1	Msg ID	0x13

CONFIDENTIAL

Query Information

The client can query the properties of the upgrade server with this command.

Table 11 - Query Information

Size	Field	Description
1	Msg ID	0x10

This command returns a status message with the following parameters:

Table 12 - Query Status Parameters

STATUS parameters

Size	Field	Description
1	version	Upgrade protocol version information and flags Bits : 0..3 : Protocol version 0x0 - v1.0 0x1 - v1.1 0x2 - v1.2 (this version) 0x3 - 0xF (RFU) Bits : 4..7 : RFU
1	capabilities	Capabilities flags Bit : 0 : Stream Mode 1 - Supports write buffer stream protocol[1] 0 - not supported Bit : 1 : Dual write buffer support 1 - Supports dual (ping-pong) buffers[1] 0 - Single buffer Bit : 2 : Resizable write buffers 1 - write buffers can be resized 0 - write buffers are fixed Bits 3..7 : RFU *Note:* *1. For BLE-GATT Profile:* • *If the profile does not support the data stream protocol the client must use the simpler (slower) method over the Mailbox.* • *If the profile supports dual buffers, data transfers can overlap with programming otherwise the client must serialize to a single buffer. (**As of this writing dual buffer support is not supported**)*
2	board_id	Vendor specific board identifier - only understood by the client
1	buffer_size	Write buffer size in 256 byte pages (8 = 4 KB).
1	erase_size	Erase sector size in 256 byte pages (16 = 4 KB)
1	flash_type	Flash type (see FLASH_TYPE_XXXX defines) - informational
2	capacity	Flash capacity in 16 KB units (max 1 GB). The inactive partition is exactly half of this.
2	part_size	Partition size in 16 KB units
3	nvds_offset	This is the upper 3 bytes of the offset of the NVDS area in the active partition. The offset should be left shift by 8 before using it in

CONFIDENTIAL

		commands (read or copy). Note that the inactive partition NVDS is unknown and not reported.
1	nvds_size	The size of the NVDS area in kilobytes (max 255 KB)
1	active_status	Bits 0..3 Status The following enumerations indicate the partition status 0x0 : reserved 0x1 : The partition is not fully programmed[1] 0x2 : The partition has been programmed 0x3: The partition is a first boot[2] 0x4 : The partition is good 0xF : The partition has been marked bad Bit 4..7 RFU *Notes:* 1. *It is not possible to know if a partition is completely erased unless an erase verify operation is performed.* 2. *The first boot status applies only to the active partition*
1	inactive_status	The upgrade status of the inactive partition (see active_status)
4	debug	Debug information (reserved for internal use only)

CONFIDENTIAL

Query Version Information

The client can query version information of the device. If the device does not include the Device Information Service the client can use this method as an alternative.

Table 13 - Query Version Information

Size	Field	Description
1	Msg ID	0x11

This command returns a status message with the version information set by the device. The maximum version length is 255 bytes. The actual version information is limited by the transport limits for a status message (See Message size limits).

Table 14 - Status Message

STATUS parameters

Size	Field	Description
N	Version information	Version information. A string or data structure defined by the versioning method. If a string is provided, the buffer may or may not contain a NULL terminator. If the device has not set the version information, this field will be empty.

Write Data Start

The client sends data to the write (staging) buffers as a sequence of messages. The first message is a start message followed by one or more continuation messages. The last fragment in the continuation starts programming. A start message marked as the only fragment also triggers the programming operation. Writes can only be issued for the inactive partition. Write operations cannot be issued when there are outstanding copy or erase operations. Writes assume the sectors have been erased prior.

Table 15 - Write Data Start

Size	Field	Description
1	Msg ID	0x20
1	flags	Write flags Bit 0 : Buffer selection (if dual buffer support) 0 - Fill Buffer 0 (default) 1 - Fill Buffer 1 Bit 1..7 : RFU (set to zeros)
4	offset	Starting offset into the inactive partition where the write buffer will be written to.
2	length	Length of all bytes to be written to the partition
1	count	Count of fragments (1-255) to send (this message is fragment 0). A count value of 0 indicates this buffer contains all the data for the operation and is considered the last fragment.
1	req_seq	Client specific, unique, per-request sequence number (returned in status and completion messages) to track the overall write operation.
<N>	data	Write data up to the remaining length of the first fragment

A status message is issued on the reception of all or missing (a timeout) sequences with parameters shown in the following table.

CONFIDENTIAL

Table 16 - Write Data Status Parameters

STATUS parameters		
Size	**Field**	**Description**
1	req_seq	Request sequence copied from the start message
1	seq_good	The highest in-order received sequence number. Can be checked if status is MISSING_FRAG. This is only used if retransmit is supported.

If the status reported is PENDING the parameters/data are accepted and the programming operation has commenced. All other statuses are considered errors and no completion message is expected. The client can wait for a completion message or queue data to the other buffer (if dual buffer mode is supported, **at this time dual buffer mode is not supported**). Any transfer to a buffer that is being programmed will be rejected with an error.

In stream mode, all reception status indications are deferred. This allows the client to stream the start followed by continuation messages without pausing for status messages. Since it is possible that any message or even all messages including the start message could be lost the status check at the end of the sequence must be checked with a timeout.

The following table shows the parameters for the completion message:

Table 17 - Write Data Completion Parameters

COMPLETION parameters		
Size	**Field**	**Description**
1	req_seq	Request sequence from start message

Write Data Continuation

This is a continuation message to deliver a write data fragment. The write buffer offset is required since the start message also delivers data into the buffer. The next continuation message should start where the previous message left off. Also, in the event a fragment is re-sent the buffer offset ensures that the data is filled at the correct location in the write buffer. See Message size limits for the upper limit of a fragment.

CONFIDENTIAL

Table 18 - Write Data Continuation

Size	Field	Description
1	Msg ID	0x21
1	req_seq	Sequence number for this request. Must match the write start message.
1	seq	Fragment sequence number (1-255). The Write Start message is always sequence 0.
2	buff_offset	Starting offset into the write buffer for the following data
<N>	data	Data up to the remaining length of this fragment

This message uses the same status parameters as Write Data Start.

In stream mode, all reception status indications are deferred. This allows the client to stream continuation messages without pausing for status messages. A final check of status with a timeout is required at the end of the sequence.

Copy Data

This command can copy a single region from the active partition to the inactive partition. This command is useful for copying NVDS data to the inactive partition without having the client read out the data. The inactive partition is assumed to be erased.

Table 19 - Copy Data

Size	Field	Description
1	Msg ID	0x22
1	flags	Copy flags Bit 0 : Copy check 0 : no copy checks during the copy 1 : perform data checks during copy Bit 1..7 : RFU (set to zeros)
4	dest_offset	The destination offset in the inactive partition to copy to
4	src_offset	The source offset in the active partition to copy from
4	length	Length of data to copy.

CONFIDENTIAL

The status message returns a status of PENDING if the parameters are accepted and the copy operation has started. A completion message with the final results of the copy is provided. A failed copy check returns a status of WR_FAILED in the completion.

Last Write Status

This message retrieves the last write status. This status can be used to determine the cause of a status timeout during a stream mode write buffer sequence.

Table 20 - Last write status

Size	Field	Description
1	Msg ID	0x23

The message returns a status message with the following parameters:

Table 21 - Status message

STATUS parameters		
Size	**Field**	**Description**
1	req_seq	Request sequence tag copied from the start message. In stream mode if the request sequence reported here is different than the expected request sequence then the write start message was likely dropped
1	seq_good	The highest in-order received sequence number from the last req_seq tagged sequence. In stream mode this value can determine if any intermediate fragments (WR CONT messages) were dropped
1	f_count	Fragment count. In stream mode this is only valid if the start message was received.

Read Data

The client can read data in either partition. This command is not optimized to read large amounts of data and intended for small extractions useful in debug or inspection (ex: inspecting NVDS).

CONFIDENTIAL

Table 22 - Read Data

Size	Field	Description
1	Msg ID	0x30
1	flags	Read flags Bit 0 : Partition selection 0 - Inactive Partition (default) 1 - Active Partition Bits 1..7 : RFU (set to zeros)
4	offset	Offset in the partition to read
2	length	Length of data to read (see Message size limits)

This command returns a status message with the following parameters:

Table 23 - Read Data Status Parameters

STATUS parameters		
Size	**Field**	**Description**
N	Data	Data read from the offset to (offset + length) (see Message size limits)

Get Programmed State

The client can obtain CRC or erase state information across multiple block ranges.

Table 24 - Get Programmed State

Size	Field	Description
1	Msg ID	0x31
1	flags	CRC/ Erase Verify flags Bit 0 : Partition selection 0 - Inactive Partition (default) 1 - Active Partition Bit 1 : Mode 0 - CRC calculation 1 - Erase state Bits 2..7 : RFU (set to zeros)
N	range_desc	Range descriptors repeated <N> times (see below)

CONFIDENTIAL

Table 25 - Range Descriptor

Range descriptor		
Size	**Field**	**Description**
4	Offset	Offset in partition
4	Length	Length of data to check (32 KB max for CRC)

For Bluetooth LE the maximum number of range entries is limited by the maximum command parameter size (See Message size limits).

This command returns a status of PENDING if parameters are accepted and the operation has been scheduled. A final command completion message is issued with the following parameters:

Table 26 - Programmed State Status Parameters

STATUS parameters		
Size	**Field**	**Description**
4 * N	CRC or erase state for range repeated N times	If CRC : Bits 0..31 : The 4-byte CRC value If erase state: Bits 0..1 : Erase state : 0 = reserved 1 = erased 2 = not erased 3 = reserved Bits 2..31 : RFU (See Message size limits)

CONFIDENTIAL

Erase Sectors

The client can erase sectors in one or more regions in the inactive partition. The client can optionally request an erase verification step that can report whether any region contains a non-erased byte. This is reflected in the command completion status as NOT_ERASED. Erase operations cannot be issued if there are pending write/erase/copy operations.

Table 27 - Erase Sectors

Size	Field	Description
1	Msg ID	0x40
1	flags	Erase flags Bit 0 : Erase Verify 0 - Do not perform erase verify (default) 1 - Perform erase verify Bit 1..7 : RFU (set to zeros)
N	sector_desc	Sector descriptor repeated <N> times (see below)

Table 28 - Erase Sectors Descriptor

Erase Sectors Descriptor		
Size	**Field**	**Description**
4	offset	Offset in partition
4	sector_count	Count of sectors to erase starting at offset

This command returns a status message with no parameters. If the status is PENDING, the erase parameters have been accepted and the erase operation is pending. Erase operations cannot be overlapped.

Set Upgrade Status

This command sets the upgrade status for the inactive or active partition. The PROGRAMMED status is applied only to the **inactive** partition in order to allow a first-boot restart where the newly programmed partition is swapped as the active partition. The GOOD status only applies to the **active** partition that is operating in first boot mode. Marking an active partition that is already GOOD returns an error. The client must determine whether the new image operating in first boot mode qualifies as a good image and then and only then set this status. An **inactive** partition CANNOT be marked GOOD. The BAD status can only be applied to an **inactive** partition. In order to set a BAD status, the client must reboot the system into the last known GOOD partition and mark the inactive partition. This status can be used to quickly mark an upgraded image in the inactive partition that has not properly booted into the first boot stage.

CONFIDENTIAL

Table 29 - Set Upgrade Status

Size	Field	Description
1	Msg ID	0x50
1	status	Partition status to set 0x0 - Reserved 0x1 - Reserved 0x2 - Set the ***inactive*** partition as PROGRAMMED 0x3 - Set the ***active*** partition as GOOD 0xF - Set the ***inactive*** partition as BAD

This message returns a status message with no parameters. The status will indicate whether the desired status could be applied to the targeted partition.

Reboot

This command allows the client to remotely reboot the system to revert to the last known GOOD partition or validate the first boot of a new upgrade.

Table 30 - Reboot

Size	Field	Description
1	Msg ID	0x51
1	type	Reboot type 0 - Normal reboot 1 - Perform first boot[1] of the inactive partition *Note:* *1. The first boot reboot type should only be requested if the inactive partition has been marked as PROGRAMMED (see Set Upgrade Status).*

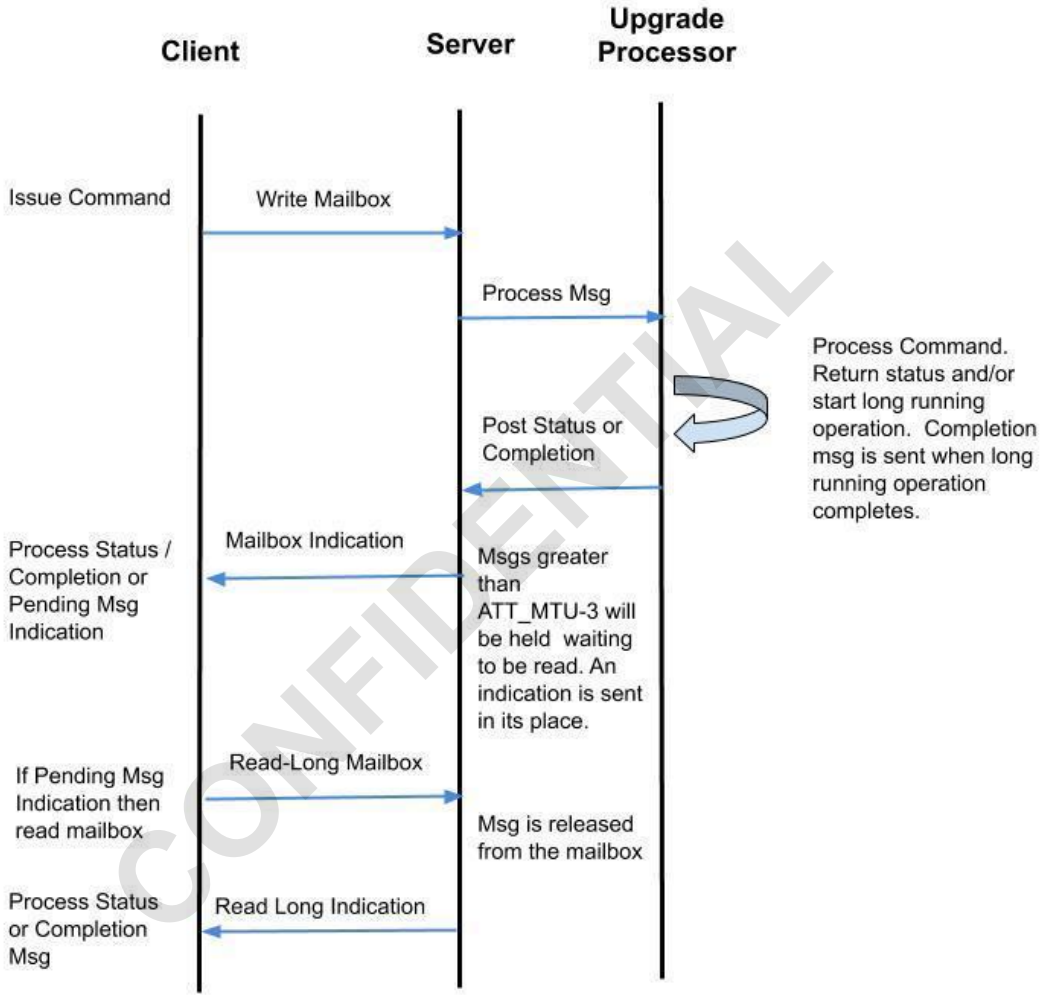
The reboot returns a status with no additional parameters, however the status may not arrive before the system reboots.

Message Flows

The following section provides flow diagrams for commands, status, completions and data.

CONFIDENTIAL

Control



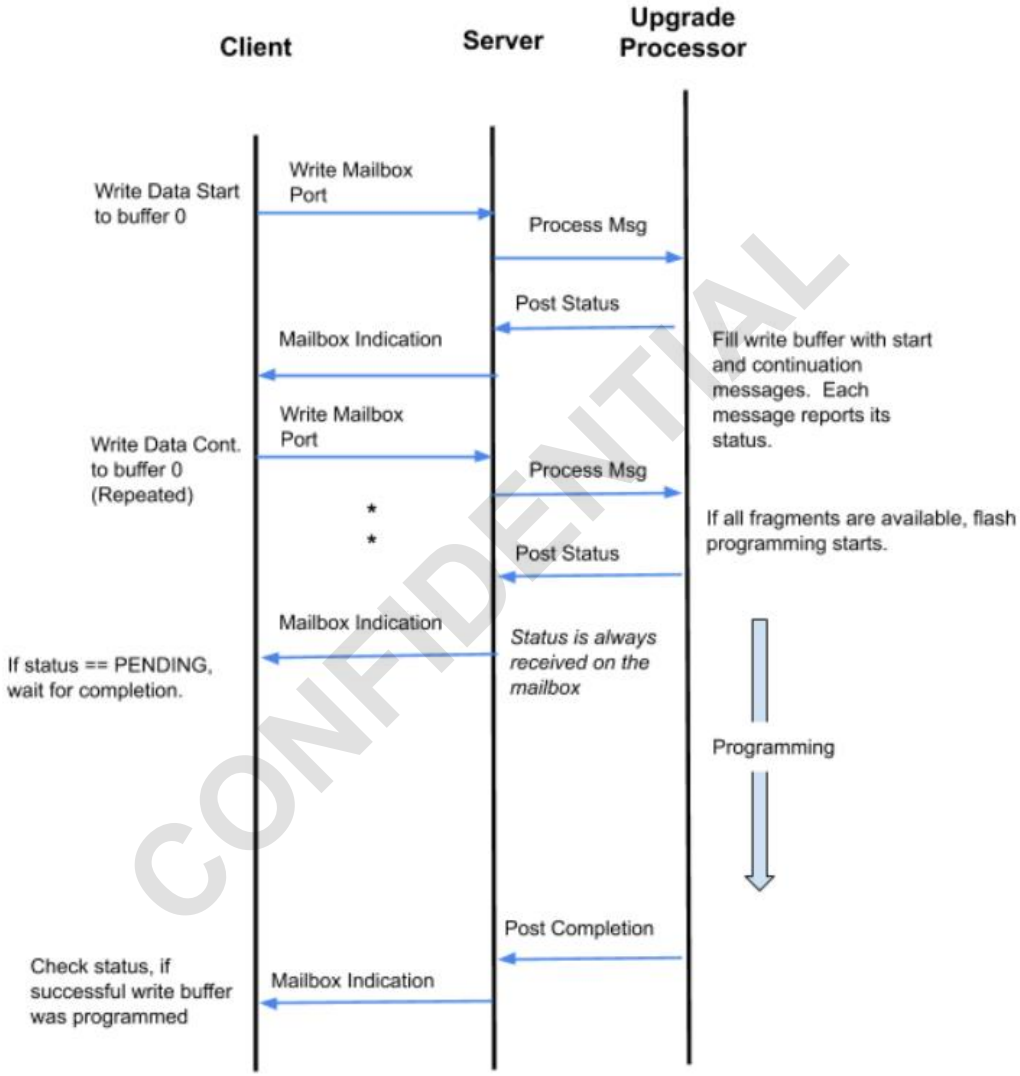
Command / Status / Completion Flow

Figure 5 - Message Flow Diagram

CONFIDENTIAL

Write Data

This is the default flow that uses the mailbox port to transfer data to the flash write buffers. All start and continuation messages report status as mailbox indications. The client must wait for the status before issuing additional messages. This increases overhead but provides reliable and easily debuggable message sequencing.



Write data flow for mailbox port

Figure 6 - Mailbox Port Write Data Flow

CONFIDENTIAL

Write Data Stream Mode

This flow uses the bulk port to stream write start and write continuation messages to fill the flash write buffer. This flow uses write-with-no-response GATT procedures and the payload of messages is limited by the negotiated MTU size in addition to the characteristic size . In this flow all intermediate status messages for each start and continuation message regardless of success or failure are omitted with the exception of the final message (start or continuation) that results in programming (i.e. the buffer is filled and ready for programming). In this mode the entire sequence of messages is considered one request and a final status is reported. The status is always reported on the mailbox port. In some client platforms and under certain conditions, the Bluetooth LE stack may optionally drop GATT write-no-response messages that do not require confirmation. This introduces the possibility of one or more (and possibly all) messages in a sequence to be dropped. This requires the client application to employ a timeout when expecting a status at the end of a buffer fill sequence. A client can expect a final status that indicates programming has begun or receive no status at all (timeout) indicating that one or more messages in the sequence was dropped including the possibility of dropping the start message. This is illustrated below:

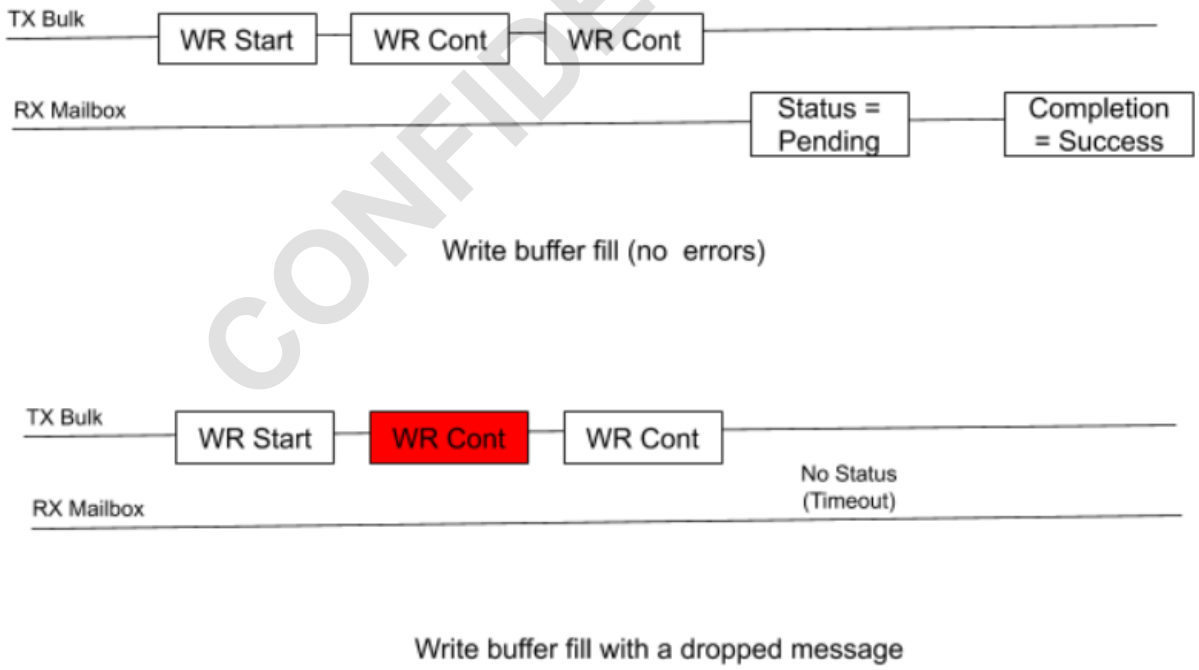


Figure 7 - Dropped Message in Write Buffer

The use of a timeout is required in the event the entire sequence is dropped. The timeout should be selected based on the number of messages (start + continuations) multiplied by the

CONFIDENTIAL

supervision timeout. For example given a typical 100 ms link supervision timeout and 5 messages (1 start and 4 fragments), the timeout for the status should be set to at least 500 ms.

In the event of a timeout the application can retry the buffer write sequence or alternatively switch to writing via the mailbox. The “Last Write Status” command can be issued to get the last write stream status to determine the cause of the timeout.

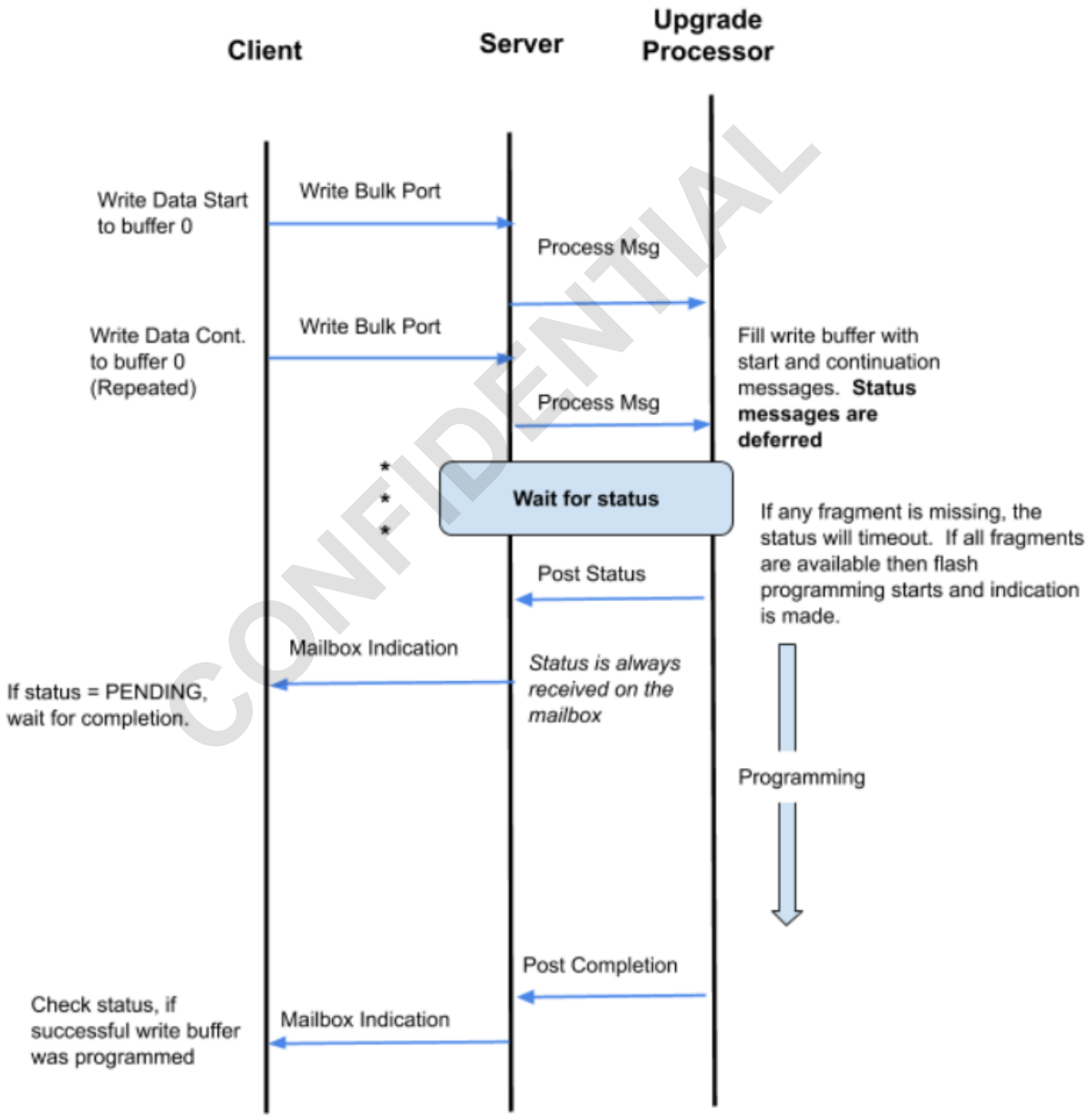


Figure 8 - Bulk Port Write Data Flow

CONFIDENTIAL

Developers Guide

Installing the profile

The following code snippets can be found in the OTA application initialization module `app_otaps.c`. The following code shows the installation of the profile with default example settings:

```
    req->operation   = GAPM_PROFILE_TASK_ADD;
    req->sec_lvl     = PERM(SVC_AUTH, NO_AUTH);
    req->prf_task_id = TASK_ID_OTAPS;
    req->app_task    = TASK_APP;
    req->start_hdl   = 0;

    // Set parameters to pass to the added service
    struct otaps_cfg *cfg = (struct otaps_cfg *)req->param;
    cfg->wr_buffer_size = 2048;   // adjustable to
    cfg->board_id       = 0xaa55; // Vendor specific
    cfg->fboot_timeout  = 3000;   // 30s (1 TU = 10MS)
    cfg->ver_info       = VERSION_STRING;
    cfg->ver_length     = VERSION_STRING_LEN;
    ke_msg_send(req);
```

The installation of the profile is similar to other profiles in the SDK. The OTA configuration data is defined in the `otaps_cfg` structure. This structure is passed in the `GAPM_PROFILE_TASK_ADD_CMD` command.

Board Identifier

The board identifier is specific to the customer's design and only understood by the client profile. This value can be hard coded by the application or read from tags stored in OTP.

Write Buffer Size

This value indicates the maximum size of the write buffer to allocate. The recommended value is at least 256 bytes. The application must determine a suitable value given its memory constraints.

First boot timeout

If this value is non-zero, the profile will start a first boot watchdog when the active partition's first boot is detected. The time value is defined in scheduling time units (10 ms). The client must connect and send any message (i.e. Query Information) to stop the watchdog. Failure to do so will cause an automatic system reboot and the system will boot from the last known good partition.

CONFIDENTIAL

Version String

The application can supply optional version information that can be passed to the OTA client. The example shows the version information set to a human readable version string. This option is provided for systems that may not implement the standard Bluetooth Device Information Service (DIS).

Changing the Service UUID

The UUID value is a constant value that requires a modification to the profile source code. This is not run-time configurable. Locate the profile source module: `profiles/otap/otaps/otaps.c`.

The constant definition is shown below:

```
static const uint8_t otaps_uuid[ATT_UUID_128_LEN] = OTAP_SERVICE_UUID;
```

Where the array is defined below showing the current default value:

```
#define OTAP_SERVICE_UUID {0x62, 0x90, 0x5c, 0x36, 0xb7, 0xc4, 0xdc, 0x8f, 0x82,
0x41, 0xb5, 0x75, 0x39, 0x8d, 0xe7, 0xa0}
```

Configuring the Flash Size

The location of NVDS is configured at firmware build time and is located below the end of the flash region defined by linker scripts and the environment variable: `FLASH_SIZE`. This region is defined at build time and is less than or equal to the actual physical flash device. In a dual image configuration the NVDS location must reside within an upgrade partition. A smaller flash region size that is at least half the physical device size must be configured. For example, a 4 Mbit (512 KB) device with dual image support should configure the firmware flash region size to 2 Mbits (256 KB) to accommodate both the image and NVDS region in a single partition. The build variable: `FLASH_SIZE=0x40000` would configure this 2 Mbit partition (256 KB).

Cleaning Upgrades

When reflashing devices using the developer tools (openocd) a flash image resident in the inactive partition (bank 1) may take precedence over the new image flashed via openocd in

CONFIDENTIAL

bank 0. This will cause an older image to boot. To resolve this, use the build variable `ERASE_UPGRADE_DATA=1`. This will ensure that the upgrade header in bank 1 is cleared.

CONFIDENTIAL



CONFIDENTIAL

ATMOSIC TECHNOLOGIES - DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2019-2021 by Atmosic Technologies. All rights reserved.

www.atmosic.com